

```

1  ( *
2
3      CS51 Lab D
4      Improving Debugging Skills
5  *)
6  (*=====
7  Readings:
8
9      This lab has no prerequisite textbook reading.
10
11 Objective: In this lab, you'll improve your debugging skills by
12 applying fundamental debugging ideas to improving code.
13
14     In the exercises that follow, some functions may have bugs, so that
15     their behavior may not match the intended behavior described in the
16     comments. Your job is to find and fix all of the bugs.
17  =====*)
18
19  (*=====
20  Part 0: Important aspects of debugging
21
22  You may not have thought explicitly about the debugging process, but
23  doing so can provide you with valuable skills in the process. Here are
24  some of the major aspects of the debugging process.
25
26      Identification
27
28          Read error messages in detail. They often provide not just the
29          nature of the error, but an approximate location.
30
31          Set up unit tests for individual functions. Unit tests can
32          identify bugs in your code by finding cases that don't match
33          the behavior you intended. Try to specify unit test cases that
34          cover all of the important paths through the code. A good
35          technique is to put the unit tests in a separate file that
36          references the file with the functions to be tested. Then,
37          whenever you make changes to the functions, you can rerun the
38          test file to make sure that you haven't introduced bugs in
39          previously working code.
40
41      Localization
42
43          When you first identify a bug, you may not know where in the
44          code base the bug actually lives. You'll need to localize the
45          bug -- finding its location in the code base.
46
47          In tracking down problems in larger codebases, eliminate
48          portions of the code to generate the minimal codebase that
49          demonstrates the problem. Breaking the code into smaller parts
50          can allow localization to one of the parts, as they can be
51          unit-tested separately.
52

```

Simplification

When confronted with an error exhibited on a large instance, try to simplify it to find the minimal example that exhibits the problem.

Reproduction

Try alternate examples to see which ones exhibit the problem. The commonalities among the examples that exhibit the problem can give clues as to the problem.

Diagnosis

Verify that invariants that should hold in the code actually do, with assertions or other constructs. (The ``Absbook.verify`` function can be especially useful in verifying invariants of the arguments and return value.) Conduct experiments to test your theory of what has gone wrong.

Correction

Generate git commits to save a version of the code so that you can confidently make changes to the code while you are experimenting, knowing that you'll be able to return to earlier versions.

Maintenance

Code that was once working can become buggy as changes are made either to the code itself or to code that it uses. It's thus helpful to retest code when changes are made to it or its environment. Fortunately, unit test files are ideal for this process. Rerunning the unit tests liberally allows us to verify that working code hasn't regressed to a buggy state. (The process is referred to in the literature as "regression testing" for this reason. See https://en.wikipedia.org/wiki/Regression_testing.)

*)

(*=====

Part 1: Some finger exercises in debugging

In this part, we'll provide implementations of a few simple functions. These functions may or may not have bugs in them. You'll proceed through five steps:

1. Read the function definition, including the top-level comment, to give you an idea of what the function is intended to do.
2. Write a full set of unit tests for each of the functions in the file ``part1_tests.ml``. The comments introducing each function may give information about the intended behavior of each function.

```

107     *** Do not skip this step. ***
108
109 3. Once you've written all of the unit tests, compile and run the
110    unit tests
111
112     % ocamlbuild -use-ocamlfind part1_tests.byte
113     % ./part1_tests.byte
114
115 4. For each of the functions, find a value for the function's
116    argument on which the function fails, if there is one. (If
117    you've built your unit tests well, they should uncover such a
118    value directly.) Record the failing input by let-defining the
119    corresponding `_fails_on` value to be `Some v` where `v` is the
120    failing input if there is one, or `None` if the function is not
121    buggy.
122
123 5. Revise each function to eliminate any bugs that you found. While
124    you're at it, you should probably deal with any warnings that
125    arise as well.
126
127 We've done the first of these exercises, the `abs` function, for you
128 to give you the idea.
129
130 1. ORIGINAL VERSION
131
132 (* abs x -- Returns the absolute value of the integer `x` *)
133 let abs x =
134     if x < ~-1 then ~- x else x
135
136 2. UNIT TESTS (These would be added to `part1_tests.ml`.)
137
138 unit_test (abs 0 = 0) "abs zero";
139 unit_test (abs 1 = 1) "abs one";
140 unit_test (abs max_int = max_int) "abs maxint";
141 unit_test (abs min_int = min_int) "abs minint";
142 unit_test (abs (-0) = 0) "abs zero";
143 unit_test (abs (-1) = 1) "abs neg one";
144 unit_test (abs (-max_int) = max_int) "abs neg maxint";
145 unit_test (abs (-min_int) = min_int) "abs neg minint";
146
147 3. COMPILE AND RUN
148
149 % ocamlbuild -use-ocamlfind part1_tests.byte
150 % ./part1_tests.byte
151 ...
152 abs zero passed
153 abs one passed
154 abs maxint passed
155 abs minint passed
156 abs zero passed
157 abs neg one FAILED      <-- This'll be helpful!
158 abs neg maxint passed
159 abs neg minint passed
160 ...

```

```

161     - : unit = ()
162
163     4. FAILING INPUT
164
165     let abs_fails_on = Some (-1)
166
167     5. REVISED VERSION
168
169     (* abs x -- Returns the absolute value of the integer `x` *)
170     let abs x =
171       if x < 0 then ~- x else x
172   *)
173
174   (* ..... *)
175   (* last_element lst -- Returns the last element of `lst` as an option;
176     `None` if there is no last element *)
177   let rec last_element lst =
178     match lst with
179     | [] -> None
180     | [x] -> Some x
181     | _ :: tail -> last_element (List.tl tail)
182
183   let last_element_fails_on = failwith "last_element_fails_on not defined"
184
185   (* ..... *)
186   (* sum_to_n n -- Returns the sum of integers from 1 to `n` *)
187   let rec sum_to_n n =
188     if n = 0 then n
189     else n + sum_to_n (n - 1)
190
191   let sum_to_n_fails_on = failwith "sum_to_n_fails_on not defined"
192
193   (* ..... *)
194   let describe_list lst =
195     match lst with
196     | [] -> "Empty list"
197     | _ :: _ -> "Multiple list"
198     | [x] -> "Singleton list"
199
200   let describe_list_fails_on = failwith "describe_list_fails_on not defined"
201
202   (* ..... *)
203   (* one_empty lst -- Returns `true` if and only if `lst` is a list
204     containing *exactly* one element, where that element is itself the
205     empty list. (For example, `one_empty [[]] = true` but
206     `one_empty [] = false` and `one_empty [[1]] = false`.) *)
207   let one_empty lst =
208     match lst with
209     | [] -> false
210     | [x] ->
211       match x with
212       | [] -> true
213       | _ -> false
214     | _ :: _ -> false

```

```

215
216 let one_empty_fails_on = failwith "one_empty_fails_on not defined"
217
218 (*=====
219 Part 2: Debugging set operations
220
221 In this part, you will apply your debugging skills to debugging an
222 implementation of set operations (union, intersection, etc.).
223
224 *****
225 In this lab, sets of integers will be represented as `int list`s
226 whose elements are in *sorted order* with *no duplicates*. All
227 functions can assume this invariant and should deliver results
228 satisfying it as well.
229 *****
230
231 To get you started on debugging, we've placed a few unit tests for
232 some of the functions in the file `part2_tests.ml`. Compile and run
233 these tests to see how the functions are working so far.
234
235 % ocamlbuild -use-ocamlfind part2_tests.byte
236 % ./part2_tests.byte
237
238 What do you notice? Does this give you an idea on where to start
239 debugging?
240
241 =====
242 Part 2A: Some utilities for checking the sorting and no-duplicates
243 conditions.
244 *)
245
246 (* is_sorted lst -- Returns `true` if and only if `lst` is a sorted
247    list *)
248 let is_sorted (lst : 'a list) : bool =
249   lst = List.sort Stdlib.compare lst ;;
250
251 (* dups_sorted lst -- Returns the number of duplicate elements in
252    `lst`, a sorted list of integers. For example
253
254       # dups_sorted [1;2;5;5;5;5;5;5;6;7;7;9] ;;
255       - : int = 6
256
257 *)
258 let rec dups_sorted (lst : 'a list) : int =
259   match lst with
260   | [] -> 0
261   | [_] -> 0
262   | first :: second :: rest ->
263     if first = second then 1 else 0
264     + dups_sorted rest
265
266 (* HINT: The `dups_sorted` function already fails two of the tests in
267    the testing file `part2_tests.ml`. Examine the failing cases one at
268    a time. What is it about the first case that causes it to fail? Can
    you find a simpler case that fails? What is the minimal example

```

```

269     that fails? Does that help you repair the first buggy test case?
270     Then turn your attention to the second case. Did you fix that bug
271     too, or is it still around? If it's still around, you'll need to
272     fix that too. *)
273
274 (* is_set lst -- Returns `true` if and only if lst represents a set,
275    with no duplicates and elements in sorted order. *)
276 let is_set (lst : 'a list) : bool =
277     is_sorted lst && dups_sorted lst = 0 ;;
278
279 (*=====
280 Part 2B: Set operations -- member, union, and intersection
281
282 Below we provide code for computing membership, intersections, and
283 unions of sets represented by lists with the stated invariant.
284
285 Check out the unit tests for these in `part2_tests.ml`. Augment the
286 tests until you're satisfied that you've fully tested these functions,
287 making any needed changes as you go.
288
289 We'll test them further on larger examples in the next part, Part
290 2C. *)
291
292 (* member elt set -- Returns `true` if and only if `elt` is an element
293    of `set` (represented as above). Search can stop early based on
294    sortedness of `set`. *)
295 let rec member elt set =
296     match set with
297     | [] -> false
298     | hd :: tl ->
299         if elt = hd then true
300         else if elt < hd then false
301         else member elt tl ;;
302
303 (* union set1 set2 -- Returns a list representing the union of the
304    sets `set1` and `set2` *)
305 let rec union s1 s2 =
306     match s1, s2 with
307     | [], [] -> []
308     | _hd :: _tl, [] -> s1
309     | [], _hd :: _tl -> s2
310     | hd1 :: tl1, hd2 :: tl2 ->
311         if hd1 < hd2 then
312             hd1 :: union tl1 s2
313         else
314             hd2 :: union tl2 s1 ;;
315
316 (* HINT: This function has no tests in the testing file. Maybe you
317    should add some. *)
318
319 (* intersection set1 set2 -- Returns a list representing the
320    intersection of the sets `set1` and `set2` *)
321 let rec intersection s1 s2 =
322     match s1, s2 with

```

```

323 | [], _ -> []
324 | _, [] -> []
325 | hd1 :: t11, hd2 :: t12 ->
326 |   if hd1 = hd2 then hd1 :: intersection t11 t12
327 |   else if hd1 > hd2 then intersection t11 s2
328 |   else intersection s1 t12 ;;
329
330 (*=====
331 Part 2C: Scaling up the testing
332
333 The file `labD_examples` contains a couple of larger examples of sets
334 represented as lists (`example1` and `example2`). The `part2_tests.ml`
335 file contains a few tests based on these larger examples, which are
336 commented out at the moment. Uncomment them now and rerun the unit
337 tests. What do you notice?
338
339 More bugs to debug. Where do you think the problems lie? Remaining
340 bugs in the functions above? In the examples? In the tests themselves?
341
342 You're on your own to figure out what's going on and correct the
343 problems, wherever they might be.
344 *)

```